Arpan Exam Notes:

Date
Page

Unit-II

Programming for Embedded System

- AVR microcontroller are based on RISC Architecture (Reduced Instruction set computing)
- AVR microcontroller are categorized into four major groups:
  Classic, Mega, Tiny, Special Purpose

General Architecture of AVR Microcontrollers

1. common features:

1. Core Architecture
   Microcontrollers are based on Harvard Architecture (separate program & data memory)
   (i) RISC based design
   (ii) General Purpose Registers

2. Program and Data Memory
   Flash Memory, SRAM, EEPROM

3. Interrupt System ; Global Interrupt Enable (GIE) mechanism

4. Clock system

5. Power management

6. Peripherals and Features
   (i) Timers/Counters
   (ii) Analog-to-Digital Converter (ADC)
   (iii) PWM Modules
   (iv) Communication Interfaces
   (v) GPIO pin

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Architecture of Atmega 32 microcontroller

The AVR microcontroller, including Atmega 32, is designed with RISC architecture. It follows Harvard architecture with separate program memory and data memory. Its architecture can be broken down into following sections:

1. Program Memory Section

(i) Flash Program Memory
- stores the program (machine code)
- Non-volatile (doesnot erase when power is off)

(ii) Program Counter
- Holds the address of next instruction
- Automatically increments after each instruction

(iii) Instruction Register
- Holds the current instruction fetched from memory

notes.arpanadhikari7.com.np

(iv) Instruction Decoder
- Decodes Instruction
- Generates control signal for execution

2. CPU Core

(i) 32 x 8 General Purpose Registers
(ii) ALU (Arithmetic Logic Unit)
(iii) Status & control Registes

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

3. Data Memory section

(i) Data SRAM
(ii) EEPROM

4. Addressing Modes
(i) Direct Addressing
(ii) Indirect Addressing

5. Peripheral Units

(i) Interrupt Unit
(ii) SPI Unit
(iii) Watchdog Timer
(iv) Analog Comparator
(v) I/O Modules
(vi) I/O Lines

6. Data Bus
8-bit

Embedded C

- Used to develop microcontroller based applications
- Extension to C, including diff. features such as addressing I/O, fixed-point arithmetic, multiple memory addressing etc.

Features:

① Efficiency & speed
② Direct Hardware Access (Features like pointers & bitwise operators)
③ Portability (structure standardized)
④ Standardized tools

notes.arpanadhikari7.com.np

Memory Management in Embedded C

- ES works with constrained memory. So, M.M. needed.
- Three techniques:
  1. Static Memory Allocation
  2. Dynamic Memory Allocation
  3. Memory Pool Allocation

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

1. Static Memory Allocation

Static memory allocation reserves memory at compile time, meaning the memory size and location are fixed throughout the program's execution.

- No runtime overhead for allocation or deallocation.
- Suitable for systems with fixed memory requirements.

E.g. int buffer[100]; //memory allocated statically for 100 integers

2. Dynamic Memory Allocation

- Allocates memory on run time
- Uses functions like malloc(), calloc(), realloc(), free()
- suitable for system with variable and unpredictable memory requirements

E.g.
```
int number = 100;
int *dynamicBuffer = (int *) malloc (number * sizeof(int)); //Allocate
if (dynamicBuffer = NULL){
    //Handle allocation failure
}
free (dynamicBuffer); //Deallocate
```

notes.arpanadhikari7.com.np

3. Memory Pool Allocation

- Memory Pool Allocation is a hybrid approach where a block of memory is pre-allocated and divided into fixed-size chunks.
- Tasks requests chunks as needed, avoiding runtime allocation overhead and fragmentation.
- Deterministic behavior, ideal for real-time systems.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Eg. #define POOL_SIZE 10
int memoryPool[POOL_SIZE];
int *getMemoryChunk(){
static int inden = 0;
return (inden < POOL_SIZE) ? &memoryPool[inden++] : NULL;
}

Data Types in Embedded C Programming

In ES, memory and speed are limited, so choosing correct data type is important.

- Basic Data types
  - char
  - int
  - float
  - double
- Signed & Unsigned
  - signed data type
  - Unsigned data type
- Fixed width Integer
  - int8_t
  - uint8_t
  - int16_t
  - uint16_t
- Special Embedded data types
  - volatile
  - const

Derived : Array, Structures, Pointers. (stores memory address)
int arr[5]; (group data types) int *ptr;
struct sensor{ int temp; int humidity; }

1. Basic Data Types

data type	size	description
char	8 bit	stores character or single integer
int	16 bit	stores integer values
float	32 bit	stores decimal value
double	32 bit	High precision decimal
long	32 bit	stores larger integer

2. signed (default) : store both positive & negative values
Unsigned : stores only positive values (gives larger positive range)
↓
commonly used in ES because Hardware registers are non-negative.
Counter/timer donot need negv. value.

notes.arpanadhikari7.com.np

Scanned with CamScanner
Available on notes.arpanadhikari7.com.np

3 Fixed Width Integer Data Type

To ensure exact size across different microcontrollers, <stdint.h> provides fixed-width types.

Type	Size	Description
int8_t	8-bit	Signed 8 bit
uint8_t	8 bit	Unsigned 8 bit
int16_t	16 bit	signed 16 bit
uint16_t	16 bit	Unsigned 16 bit
uint32_t	32 bit	Unsigned 32 bit

4 Special Data Type for Es

(i) volatile

- Used for variable whose value may change anytime (interrupt or hardware)
- Prevents compiler optimization
- Used for hardware registers & ISR variables.

E.g. volatile uint8_t flag;

notes.arpanadhikari7.com.np

(ii) const

- values cannot be modified.
- Used for configuration constant & lookup tables.

E.g. const uint8_t maxValue = 10;

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Control structures in Embedded C Programming

1. Decision making
   - if statement (sensor threshold checking, Interrupt flag checking)
   - switch statement (Menu systems, command Processing)

2. Looping structures
   - while: (very common) - while(1) creates infinite loop
     - Used to run program forever
   - for : (delay loops, iterating loops)
   - do-while : (Executes at least once)

```
while(1){
    readSensor();
}
```

```
for(int i=0; i<10; i++){
    delay();
}
```

```
do{
    read();
}while(flag == 0);
```

3. Jump Statements
   - break (stop loop entirely)
   - continue (skip current iteration)
   - return (Exit a function & optionally return a result)

Used for exiting loops, function termination

notes.arpanadhikari7.com.np

Pointers in Embedded C programming

```
int n=10; int *ptr;
int *ptr=&n;
```

- Store memory addresses
- Used for hardware register access
- Used in drivers and interrupt handling
- Must be used carefully

Pointer allow: - direct hardware access
- efficient - memory usage
- faster execution
- ISR handling

Types of pointer:
① Null pointer `int *p = NULL;`
② void pointer `void *p;`
③ pointer to pointer `int **pp;`
④ function point `int *funcptr(){};`

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

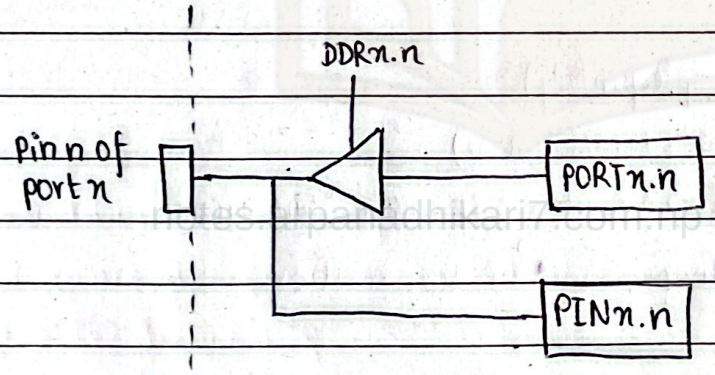
Input and Output Ports Interfacing on AVR

The total number of pins of AVR microcontroller depend on the variant of the chip we are using. Ports provide interface for input and output operations, the configuration of port must be programmed for required functionality.

Each I/O port in AVR microcontroller is associated with 3 registers. These 3 registers control behavior of port for i/p, o/p and data retrieval.

Registers

Data Register (PORTx)	Data Direction Register (DDRx)	Input Pins Address Register (PINx)
~~Configures~~ - Sets o/p pin HIGH or LOW	- configures pins of port as i/p or o/p	- Reads current state of pin



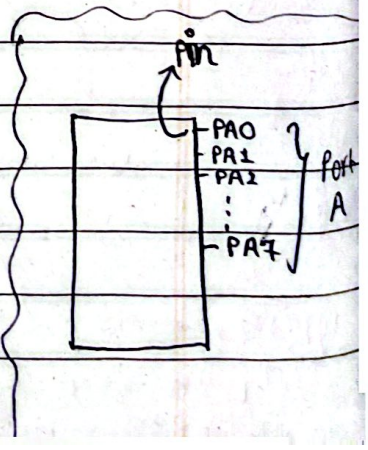


Fig. The I/O port in AVR

Scanned with CamScanner
Available on notes.arpanadhikari7.com.np

1. Data Direction Registers (DDRx)

**Purpose : Configures pins of port as either input or output.

* Features :

- These are 8-bit registers(one bit controls one pin)
- Writing 1 → Sets pin as output pin
- Writing 0 → Sets pin as input pin
- Default value after reset = 0x00 (all pins input)

E.g. DDRD = 0x00; // All port D pins are ~~output~~ input
DDRA = 0xFF; // All port A pins are output

2. Data Registers (PORTx)

Purpose : Controls logic state (HIGH or LOW) of port pins.

- These are 8 bit registers.
- Writing 1 → sets pin to logic HIGH (5V)
- Writing 0 → sets pin to logic LOW (0V)
- These registers can both be read & written to.
- Default value = 0x00 (LOW)

E.g. PORTD = 0x55; // 01010101 → Alternate pins HIGH

notes.arpanadhikari7.com.np

3. Input Pins Register (PINx)

Purpose : Read logic levels present in pins of the port

- 8 bit read-only registers
- Value of each bit represents current logic level of corresponding pin
- These registers cannot be written to

E.g. uint8_t port_value = PIND;
// Read logic level for Port D

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Timers and Counters in AVR

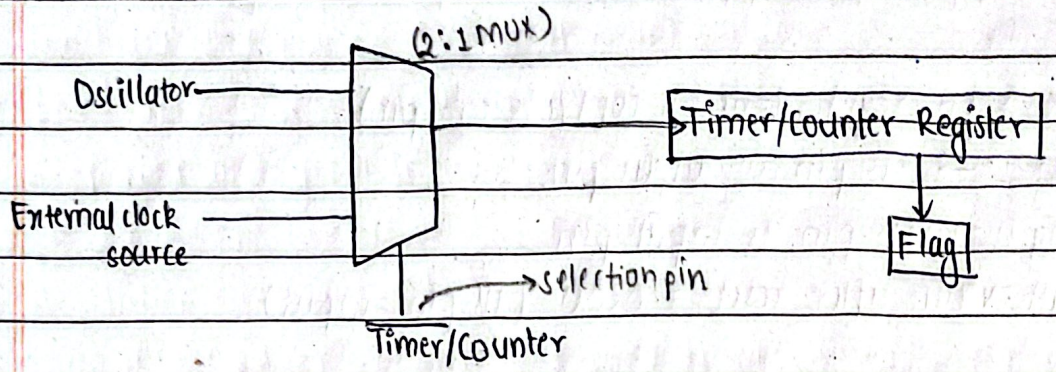


Fig. General view of Timer/counter in AVR microcontroller

- Selection pin as 0 → Timer selected (Oscillator)
- selection pin as 1 → ~~External~~ Counter selected (External clock source)

Counter

- For counter selection, set selection pin as 1
- Each time external event occurs, counter increments.

Timer

- For timer selection, set selection pin as 0.
- When used as timer, the pin is connected to a oscillator.
- For each pulse, the timer ~~increases~~ count increases

- Timer/counter register is the register that stores the current count value. E.g. TCN0, TCNT1 etc.

- Flag indicates that timer event (like overflow) has occurred.
- When counter reaches max value (255) & move to 0, overflow flag is set.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

XTAL → Crystal.

Date
Page

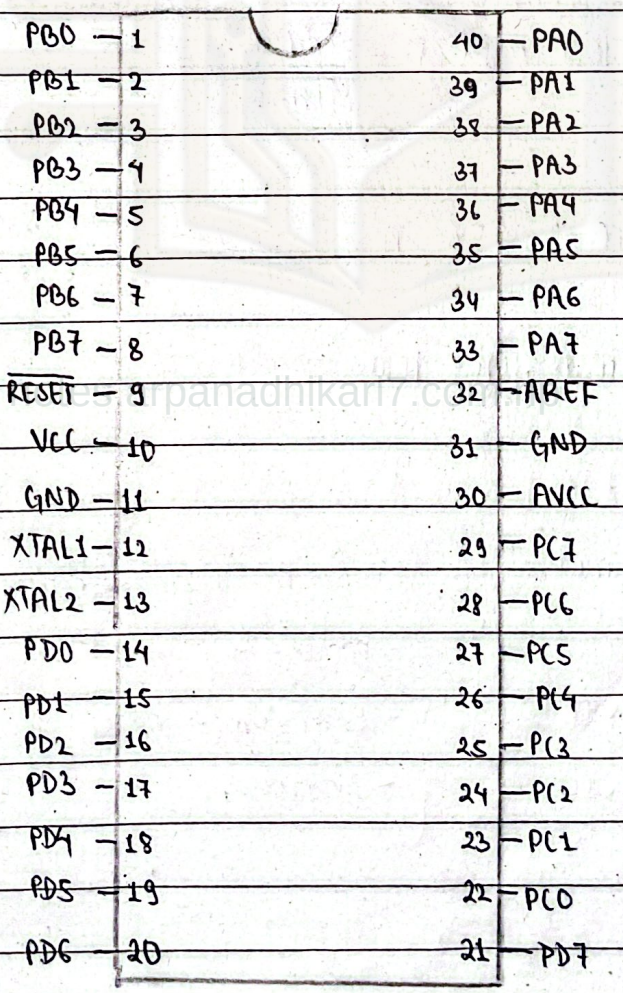
Timers in Atmega32 (Three timers/counters).

(i) Timer 0 : 8-bit timer
(ii) Timer 1 : 16-bit timer
(iii) Timer 2 : 8-bit timer

Registers associated with timer in Atmega32:

1. TCNTn (Timer Counter Register)
2. TOVn (Timer Overflow Flag)
3. ~~TENT~~ TCCRn (Timer counter control Register)
4. OCRn (Output compare Register)
5. OCFn (Output Compare Match Flag)

Atmega32 pin Diagram



adhikari-arpan/BECE_Notes

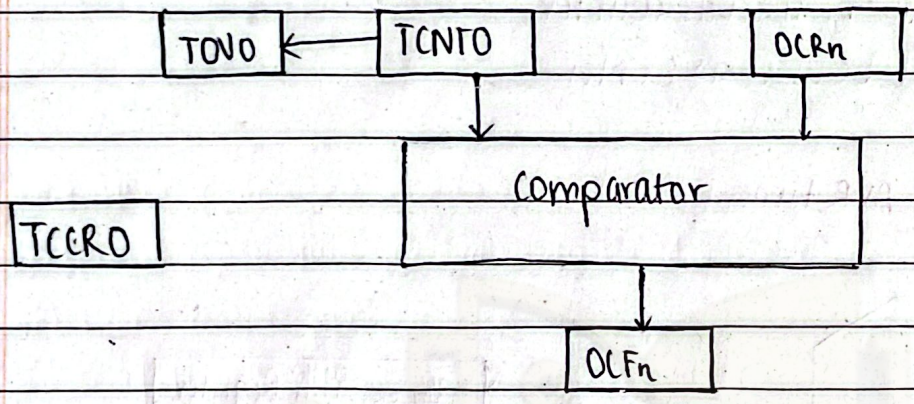
Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

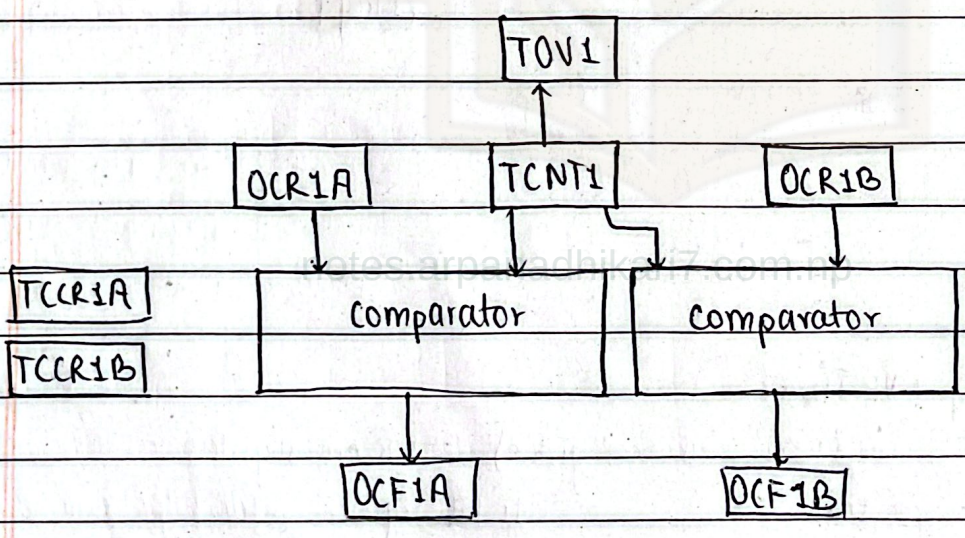
- TIFR (Timer Interrupt Flag Register)

Timers in Atmega 32

Timer 0 :



Timer 1 :



notes.arpanadhikari7.com.np

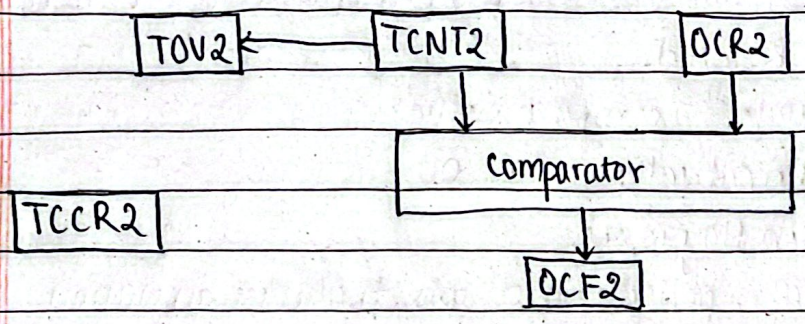
Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

e.g. f = 16MHz
Prescaler = 64.
So, $f = \frac{16}{64} = 250kHz$ $T = \frac{1}{250,000} = 4\mu s$ (counts after 4μs) { without scale $T = 0.0625\mu s$ (counts faster).
↳ more delay.

- Normal mode
- CTC mode

Timer 2: (8bit Timer)



(CPU → Prescaler → Timer/counter)

Prescaler:
Prescaler is a circuit that divides the main clock frequency before it reaches the timer/counter.
This helps the timer count slower, which makes it easier to generate longer time delays.

More Prescales ⇒ slower Time counting (counts slower) ⇒ Large delay.
(freq ↓, T ↑, counts slower).

Mode of operation of AVR Timers

Timers in AVR microcontroller can operate in different mode to perform tasks like time delay generation, event counting and waveform generation.
Mode selected using ⟶ TCCRn

Modes: (i) Normal mode
(ii) Clear Timer on compare (CTC) Mode
(iii) Phase Correct PWM Mode
(iv) Fast PWM Mode

1. Normal Mode (Overflow Mode)
- In this mode, the timer simply counts clock pulse from 0 to maximum value.
  For 8-bit timer → counts 0 to 255 For 16-bit timer → counts 0 to 65535
- When max^m value reached, timer overflows and returns to 0.
- At this moment overflow flag (TOVn) is set, & interrupt can be generated.
- Use: Generating time delay, simple timing operations

notes.arpanadhikari7.com.np

Scanned with CamScanner
Available on notes.arpanadhikari7.com.np

2. (ii) Clear Timer on Compare (CTC) Mode
- In this mode, the timer counts from 0 upto predefined value stored in OCRn (Output Compare Register).
- When the timer value equals the compare value:
  - The timer is automatically reset to 0.
  - A compare match flag is set.
- Uses : Precise time intervals, periodic interrupts, frequency generation

3. Fast PWM Mode
- In this mode, the timer generates Pulse width Modulation (PWM) signals.
- The timer counts from 0 to its maximum value continuously, when the value of timer matches with value of OCRn, the output pin changes state.
- This produces a PWM signal with adjustable duty cycle.
- Because timer only count upwards, the PWM signal is generated faster.

Uses: Motor speed control, LED brightness control, Power control systems

notes.arpanadhikari7.com.np

4. Phase correct PWM Mode
- In this mode, the timer counts up and then down between 0 and the maximum value.
- This produces a symmetrical PWM waveform, reducing signal distortion.

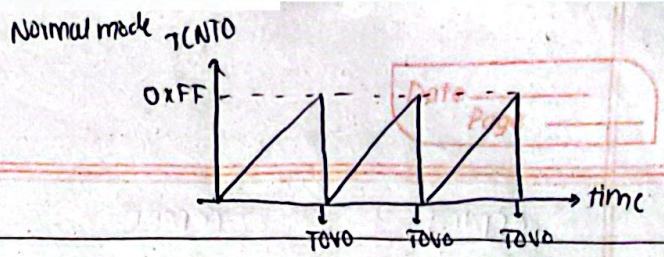
Use: High quality PWM waveform generation, Motor control, audio signal generation

adhikari-arpan/BECE_Notes

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Timer 0 Registers



1. TCNT0 (Timer/Counter Register 0)

8-bit, Holds initial value, value stored here increments with each clock pulse.

D7	D6	D5	D4	D3	D2	D1	D0

2. OCR0 Register

- Used to configure timer in CTC mode, 8 bit

D7	D6	D5	D4	D3	D2	D1	D0

3. TCCR0 Register (Controls entire operation of timer 0)

8 bit, configures mode of operation & clock source selection for Timer 0.

7	6	5	4	3	2	1	0
FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00
W	RW	RW	RW	RW	RW	RW	RW

(i) Bit 7 - FOC0 : Force Compare Match (only writable)

- Writing 1 forces a compare match immediately. (Forced compare match even if they are not equal)
- Used for generating special waveform.

(ii) Bit 6, 3 - Waveform Generation mode

WGM00	WGM01	Timer0 mode
0	0	Normal
0	1	CTC (clear Timer on Compare)
1	0	Phase correct PWM
1	1	Fast PWM

notes.arpanadhikari7.com.np

(iii) Bit 5, 4 : COM01, COM00 (Compare output mode) (used for phase correct PWM, Fast PWM).

- These bits control waveform generator.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

OC0 pin (Output ...)

COM01	COM00	Meaning
0	0	Normal port operation (OC0 not controlled by tim
0	1	Toggle OC0 pin on compare match
1	0	clear OC0 pin on compare match
1	1	set OC0 pin on compare match

(iv) Bit 2,1,0 : CS02, CS01, CS00 : Clock source select (These bits are used to select clock source)

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	clk (NO prescaling)
0	1	0	clk/8 (2^3)
0	1	1	clk/64 (2^6)
1	0	0	clk/256 (2^8)
1	0	1	clk/1024 (2^{10})
1	1	0	External clk source on T0 pin. Clock on ↓ edge
1	1	1	External clk source on T0 pin. Clock on ↑ edge

3. TIFR : Timer Counter Interrupt Flag Register

7	6	5	4	3	2	1	0
OCF2	TOV2	ICF1	OCF1A	OCF1B	TOV1	OCF0	TOV0
			(OCF1A)				

7 - Timer 2 output compare Match Flag
6 - Timer 2 overflow Flag
5 - Input capture Flag
4 - Timer 1 output compare A Match Flag
3 - Timer 1 output compare B Match Flag
2 - Timer 1 overflow Flag
1 - Timer 0 output compare Match Flag
0 - Timer 0 overflow Flag

adhikari-arpan/BECE_Notes

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

T_{delay} ; Time delay → working time (TCNT0 को value overflow हुन time)

T_{clock} → Time period → यति Time मा euta Increment हुन्छ ।

± Steps to program Timer0 in Normal Mode

1. Load TCNT0 register with initial count value.
2. Load TCCR0 register indicating mode and prescaler option to be used.
3. Keep monitoring the timer overflow flag TOV0 to see if it raised. Get out of the loop when TOV0 becomes high.
4. Stop timer by disconnecting clock source.
5. Clear TOV0 register by writing 1 to that bit of TIFR register.
6. Go back to step 1 & repeat the same.

Steps to determine initial value of TCNT0

1. $T_{clock} = 1 / F_{timer}$ (For prescaling, F_{timer} need to be calculated first)

2. Find number of counts needed. $n = \frac{T_{delay}}{T_{clock}}$

3. Initial value of TCNT0 $\simeq 256 - n$ (यो बाट suru हुन्छ ।)

4. Convert initial value into Hex & load to TCNT0.

{ Interpret Hexadecimal:
E.g. 0xCE
$= (C \times 16^1) + (E \times 16^0)$
$= 12 \times 16 + 14 \times 16$
$= 206$ }

{ Convert to Hex from decimal in calc
- mode + 4
- Enter number in Dec
- Press Hex. }

adhikari-arpan/BECE_Notes

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Q1) Assuming that XTAL = 8MHz, write a program to generate a square wave with a period of 12.5 μs on pin PB3. (Here we use no prescaler clock option).

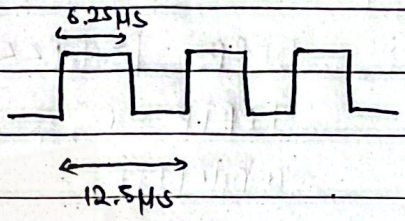
Sol^n

Finding values:

As no prescaling, $F_{timer} = 8MHz$ (same as oscillator)

So, $T_{clock} = \frac{1}{F_{timer}} = \frac{1}{8 \times 10^6 Hz} = 0.125 \mu s$

Period of square wave = 12.5 μs



So, $T_{delay} = 6.25 \mu s$

$Steps(n) = \frac{T_{delay}}{T_{clock}} = \frac{6.25 \mu s}{0.125 \mu s} = 50$

Value in TCNT0 = 256 - n

= 256 - 50

= 206 = 0xCE

Value in TCCR0:

7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	1

= 0x01

PTO

adhikari-arpan/BECE_Notes

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Program:

PB3 = 3 in avr header.

(1 << B) shift 1 3 times from last

00000001
00000010 1st
00000100 2nd
00001000 3rd

```
#include <avr/io.h>
#define F_CPU 8000000UL

void TODelay();

int main(void)
{
    DDRB |= (1<<PB3);    //Make PB3 as o/p port
    PORTB = 0;
    while(1){
        PORTB = ~PORTB;
        TODelay();
    }
    return 0;
}

void TODelay(){
    TCNT0 = 0xCE;
    TCCR0 = 0x01;
    while((TIFR & 0x01)==0);    //wait for TOV0 to roll over
    TCCR0 = 0;                  //Stop giving clock pulse (timer stop)
    TIFR = 0x01;                //Clear TOV0
}
```

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Date
Page

(ii) {50% duty cycle ⇒ signal stays HIGH for the same amount of time as it stays LOW}

Square Wave Using Timer0 (PORTB5)

```
#include <avr/io.h>
#define F_CPU 8000000UL

void T0Delay();

int main(void)
{
    DDRB |= (1<<PB5);
                                    PORTB = 0;
    while(1){
        PORTB ^= (1<<PB5);          PORTB = ~PORTB;
        T0Delay();
    }
    return 0;
}

void T0Delay(){
    TCNT0 = 0xCE;
    TCCR0 = 0x01;                   //No prescaling used.
    while ((TIFR & 0x01) == 0);
    TCCR0 = 0;
    TIFR = 0x01;
}
```

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Square wave Using Timer 1 (Port B3)

```
#include <avr/io.h>
#define F_CPU 8000000UL

void T1Delay();

int main(void){
    DDRB |= (1<<PB3);

    while(1){
        PORTB ^= (1<<PB3);
        T1Delay();
    }
    return 0;
}

void T1Delay(){
    TCNT1 = 0xFF00;
    TCCR1B = 0x01;

    while ((TIFR & 0x04) == 0);

    TCCR1B = 0;
    TIFR = 0x04;
}
```

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Polling:
```
while(1){
    check device;
    if(condition){
        perform tasks;
    }else{
    }
}
```

```
while(1){
    do other tasks;
    ISR(){
        handle event;
    }
}
```

Date
Page

AVR Interrupt Handling

Interrupt is an internal or external events that inform the CPU something need to be done/some action should be taken.

Interrupt v/s Polling

Interrupt service Routine & Interrupt Vector Table

ISR → special function or block of code that executes when interrupt occurs

IVT → Collection of address of ISR

IVT for Atmega 32:

Vector No.	Source	Interrupt	Program Address
1.	RESET	Reset	0000
2.	INT0	External Interrupt Request 0	0002
3.	INT1	External Interrupt Request 1	0004
4.	INT2	External Interrupt Request 2	0006
⋮	⋮		⋮
21.			0028

notes.arpanadhikari7.com.np

Interrupt Handling in Atmega32

1. Complete current instruction and save address of PC (Next instruction)
2. Jump to Interrupt Vector Table
3. Executes ISR (Execute & Reach RETI Instruction)
4. Return from ISR

Enabling & Disabling Interrupts

adhikari-arpan/BECE_Notes

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

AVR C-Programs

Q) Write an AVR C program to send values 00-FFh to port B.

```
#include <avr/io.h>          //standard AVR header
#include <util/delay.h>
 int main (void)
 {
    unsigned char z;
    DDRB = 0xFF;             //All pins of port B set as o/p
  while(1){
    for (z=0 ; z<=255; z++){
       PORTB = z ;
    } _delay_ms(10);
    }}
    return 0;
 }
```

Q) Write an AVR program to send hex values for ASCII characters of ~~0,1,2,A,B~~ to port B. (0,1,2,3,4,5,A,B,C,D)

notes.arpanadhikari7.com.np

```
#include <avr/io.h>
#include <util/delay.h>

 int main (void)
 {
    unsigned char z;
    unsigned char arr[] = "012345ABCD";
```


Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

```
    DDRB = 0xFF;

    while(1){
        for(z=0; z<10; z++){
            PORTB = arr[z];          //sends ASCII value
            _delay_ms(500);
        }
    }
    return 0;
}
```

Q3) AVR program to toggle all bits of PORTB 200 times.

```
#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
    unsigned char z;
    DDRB = 0xFF;
    PORTB = 0xAA;          //Initial value : 1010 1010
    while(1)
    for(z=0; z<200; z++){
        PORTB = ~PORTB;
        _delay_ms(100);
    }
    return 0;
}
```


Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Value read गर्न → Pin use.
For Port B को value read → PINB register
C PINC

Q4) AVR program to get one byte of data from port B and send it to PORTC.

```
#include <avr/io.h>

int main(void){
    unsigned char temp;
    DDRB = 0x00;          //as Input
    DDRC = 0xFF;          //as output

    while(1){
        temp = PINB;
        PORTC = temp;
    }
}
```

Q5) Write an AVR C program for LED blinking using 10ms delay. WAP that continuously sends two alternating bit pattern 0xAA & 0x55 to port B with 10 ms delay.

```
#include <avr/io.h>
#include <util/delay.h>

int main(void){
    DDRB = 0xFF;      //set as O/p
    while(1){
        PORTB = 0xAA;
        _delay_ms(10);
        PORTB = 0x55;
```

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

```
        _delay_ms(10);
    }
    return 0;
}
```

Q6) Write an AVR C program to display binary counting sequence (0-7) on LEDs connected to portc with delay of 10ms bet^n each pattern.

```
#include <avr/io.h>
#include <util/delay.h>

int main(void){
    DDRC = 0xFF;
    unsigned char z;
    while(1){
        for(z=0; z<8; z++){
            PORTC = z;
            _delay_ms(100);
        }
    }
    return 0;
}
```

adhikari-arpan/BECE_Notes

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

	PC_0
	PC_1
	PC_2
Atmega	PC_3
32	PC_4
	PC_5
	PC_6
	PC_7

Q7) Write an AVR C program to get a byte of data from PORTC. If it is less than 100, then send it to B else port D.

```
#include <avr/io.h>

int main(void){
    DDRC = 0x00;
    DDRB = 0xFF;
    DDRD = 0xFF;

    unsigned char value;
    ~~value =~~
    while(1){
        value = PINC;
        if(value < 100){
            PORTB = value;
        }else{
            PORTD = value;
        }
    } return 0;
}
```

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Timer

Date
Page

{50% duty cycle ⇒ signal stays HIGH for the same amount of time as it stays LOW}

Square Wave Using Timer0 (PORTB5)

```
#include <avr/io.h>
#define F_CPU 8000000UL

void TOdelay();

int main(void)
{
    DDRB |= (1<<PB5);
                                    PORTB = 0;
    while(1){
        PORTB ^= (1<<PB5);          PORTB = ~PORTB;
        TOdelay();
    }
    return 0;
}

void TOdelay(){
    TCNT0 = 0xCE;
    TCCR0 = 0x01;                   //No prescaling used.
    while ((TIFR & 0x01)==0);
    TCCR0 = 0;
    TIFR = 0x01;
}
```

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Square wave Using Timer 1 (Port B3)

```
#include <avr/io.h>
#define F_CPU 8000000UL

void T1Delay();

int main(void){
    DDRB |= (1<<PB3);

    while(1){
        PORTB ^= (1<<PB3);
        T1Delay();
    }
    return 0;
}

void T1Delay(){
    TCNT1 = 0xFF00;
    TCCR1B = 0x01;

    while ((TIFR & 0x04) == 0);

    TCCR1B = 0;
    TIFR = 0x04;
}
```

notes.arpanadhikari7.com.np

Scanned with CamScanner

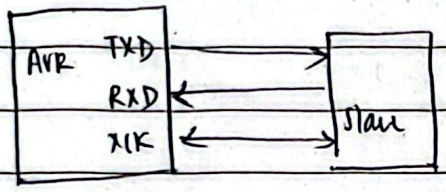
Available on notes.arpanadhikari7.com.np

serial communication in AVR. (send data in single line)

{ Baud rate register
$UBRR = \frac{F_{CPU}}{16 \times Baud} - 1$ }

AVR supports:

(i) Asynchronous (USART in UART Mode) TXD, RXD used, NO clock signal, most common
(ii) synchronous (USART) Uses clock (XCK pin)
(iii) SPI (Serial Peripheral Interface) Uses MISO, MOSI, SS, SCLK.
(iv) I²C Two wire communication (SDA, SCL)



USART communication in AVR.

AVR Atmega32 has built in USART.
~~three pins~~:
Pins used: TXD, RXD, XCK

Frame format start bit | Data bits | Parity bit | stop bit |
1 bit (Always 0) 5 to 9 bit 0 to 1 bit 1 to 2 bits.

Steps for receive data.
1. set baud rate (UBRR)
2. Enable Receiver (RXEN) (UCSRB)
3. set frame format (UCSRC)
4. Wait for RXC = 1
5. Read data from UDR

Steps for Transmit data.
1. Set Baud Rate
2. Enable transmitter
3. Set frame format
4. Wait for UDRE = 1
5. Put data in UDR

notes.arpanadhikari7.com.np

Scanned with CamScanner
Available on notes.arpanadhikari7.com.np

Q) Write a program in Atmega 32 to receive bytes of data serially & out them on Port A with baud rate of 9600, 8-bit data & 1 stop bit

Given, Baud rate = 9600 , Data bits = 8, stop bit = 1 , F_{CPU} = 8MHz.

$$\text{So, } UBRR = \frac{F_{CPU}}{16 \times Baud} - 1 = \frac{8000000}{16 \times 9600} - 1 = 51$$

```
#include <avr/io.h>
#define F_CPU 8000000UL

void USART_init(void){
    unsigned int ubrr = 51;
    UBRRH = (unsigned char)(ubrr>>8);
    UBRRL = (unsigned char) ubrr;

    UCSRB = (1<<RXEN);
    UCSRC = (1<<URSEL)|(1<<UCSZ1)|(1<<UCSZ0);
}

unsigned char USART_receive(void){
    while(!(UCSRA & (1<<RXC)));
    return UDR;
}

int main(void){
    unsigned char received_data;

    DDRA = 0xFF;
    PORTA = 0x00;
    USART_init();
    while(1){
        received_data = USART_receive();
        PORTA = received_data;
    }
    return 0;
}
```

RXC → receive complete flag.

USART Data register.

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np